AI / ML

Neural Networks Visual Guide

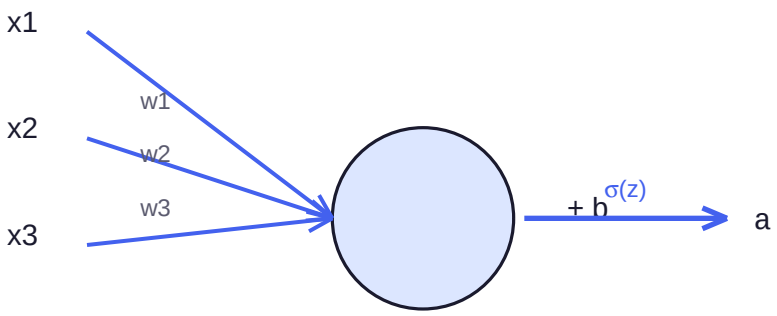
Neurons, layers, and learning —
how machines learn from examples.

ANIMATED WALKTHROUGH
Step through pages to play the animation

STEP 1/6

The Neuron: Weighted Sum + Nonlinearity

Biology-inspired math: combine inputs, squash, pass on.



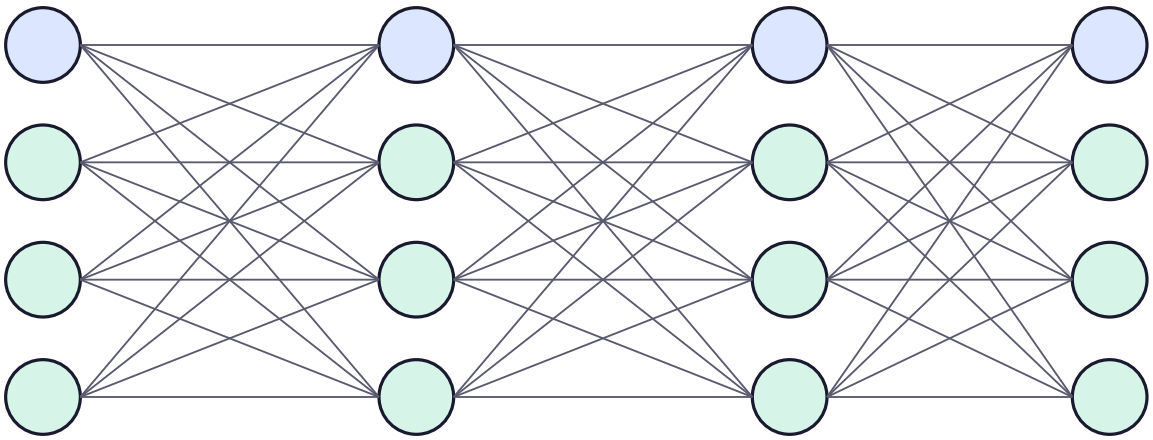
$$z = w1 \cdot x1 + w2 \cdot x2 + w3 \cdot x3 + b \;\rightarrow\; a = \sigma(z)$$

σ = activation: ReLU, sigmoid, tanh — the nonlinearity that makes depth powerful

STEP 2/6

Layers: From Pixels to Concepts

Stack neurons; each layer learns richer features.

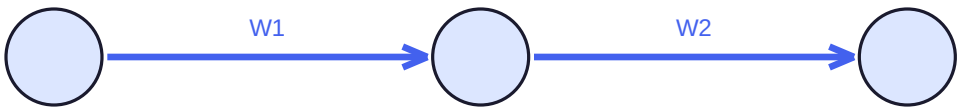


Depth builds hierarchy: pixels → edges → shapes → objects → decisions.

STEP 3/6

Forward Pass: Prediction

Data flows left to right; out pops a prediction.



Input image → multiply by weights, add biases, activate → ... → output scores:

```
[0.01, 0.02, 0.90, 0.01, …]  →  predicts '2' with 90% confidence
```

Softmax turns raw scores into probabilities that sum to 1. The highest wins.

STEP 4/6

Loss: How Wrong Were We?

One number summarizing prediction error — the thing we minimize.

True label: '2' → one-hot: [0, 0, 1, 0, 0, 0, 0, 0, 0, 0]

Predicted: [0.01, 0.02, 0.90, 0.01, …]

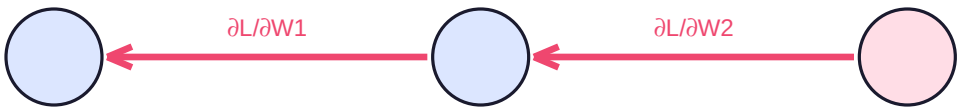
Cross-entropy loss = −log(0.90) ≈ 0.105

- Confident + correct → tiny loss. Confident + WRONG → huge loss.
- Loss is differentiable — we can compute how each weight contributed (that's next).
- Training = find weights that minimize average loss over the dataset.

STEP 5/6

Backpropagation: Blame Assignment

Chain rule from calculus, applied at scale.



Gradients flow RIGHT to LEFT — the reverse of prediction.

- Each weight gets a gradient: 'nudge me this way to reduce loss.'
- Gradient descent: $w \leftarrow w - \eta \cdot \partial L/\partial w$ (η = learning rate).
- Too big η: diverge. Too small η: train forever. Schedules + Adam help.

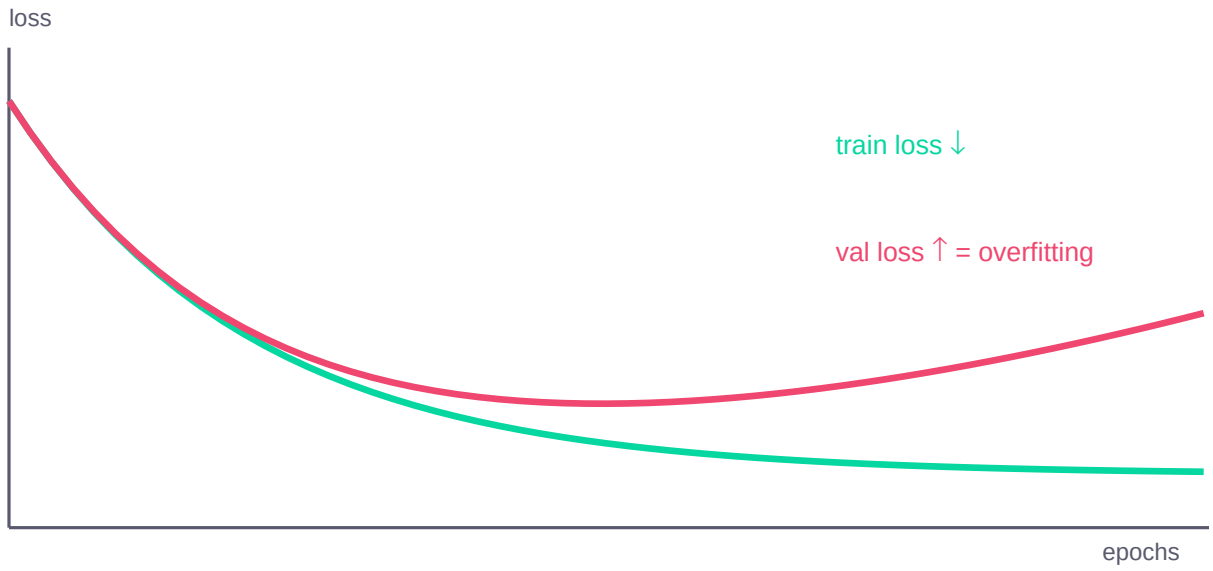
STEP 6/6

Training Loop & Overfitting

Epochs, batches, and the generalization gap.

Loop

for epoch: shuffle → mini-batches → forward → loss → backward → update



Cures: more data, regularization (dropout, weight decay), early stopping, simpler models.

STEP ✓/✓

Key Takeaways

Neural Networks — Remember

- Neuron: $z = \Sigma w \cdot x + b$, then $a = \sigma(z)$. Nonlinearity makes depth meaningful.
- Forward pass predicts; loss (cross-entropy) scores the prediction.
- Backprop sends gradients backward; gradient descent nudges weights downhill.
- Watch the validation curve — when it rises while training falls, you're overfitting.
- Depth learns hierarchy: pixels → edges → shapes → concepts.

Keep exploring

Pair this guide with the matching video walkthroughs
and hands-on labs at interstitiumlabs.dev